CS459/659
Privacy, Cryptography, Network and Data Security
Spring '26 - Assignment II

Please use Piazza for questions and clarifications.

Assignment due date: July 7, 2026, at 3:00 pm

Total Marks: 72 (32 written + 40 programming)

TA: Joseph Vendryes

TA Office hours: Thu. 3:30 – 4:30 PM on Jun. 11, 18, 25, and Jul. 2 in DC 3333E

- Submissions are to be uploaded to the appropriate LEARN dropbox as a zipped folder containing both the written and programming components.
- All questions (about the programming and written parts) should be asked **privately on Piazza under Assignment 2**.
- The TA may decide to make responses to your private questions public so that everyone benefits from knowing the same information. Questions where you need to post partial solutions or questions that describe the steganographic technique, feature sets, or code will be sanitized before the TA provides a public response.

1 Written (32 Marks)

1. **(12 marks)** Recall that HTTPS (HyperText Transfer Protocol Secure) is the secure version of HTTP, the protocol over which data is sent between your browser and the website that you are connected to. HTTPS uses Transport Layer Security (TLS) or its predecessor, Secure Sockets Layer (SSL), to encrypt the data being sent. When you access a website over HTTPS, your browser first establishes a TCP connection to the server on port 443 (the standard port for HTTPS). The client-side of the connection is on a TCP port number above 1023. To allow inbound and outbound HTTPS traffic, suppose you generate the following rule set for your packet filter firewall:

Rule	Direction	Src Addr	Dest Addr	Protocol	Dest Port	Action
A	In	External	Internal	TCP	443	Permit
B	Out	Internal	External	TCP	>1023	Permit
C	Out	Internal	External	TCP	443	Permit
D	In	External	Internal	TCP	>1023	Permit
E	Either	Any	Any	Any	Any	Deny

Table 1: Simplified Packet Filter Rule Set for HTTPS traffic

   (a) **(4 marks)** Your host in this example sits within an internet network 172.16.1.0/24 and has IP address 172.16.1.1. Someone tries to initiate an HTTPS session from a remote host with IP address 192.168.3.4 that is external. If successful, this generates an HTTPS data transfer between the remote user and the HTTPS server on your host. Additionally, assume that a user on your host tries to initiate an HTTPS session with the HTTPS server on the remote system. Four typical packets for this scenario are as shown in Table 2. Indicate which packets are permitted or denied and which rule is used in each case.

Packet	Direction	Src Addr	Dest Addr	Protocol	Dest Port	Action	Rule
1	In	192.168.3.4	172.16.1.1	TCP	443	?	?
2	Out	172.16.1.1	192.168.3.4	TCP	1234	?	?
3	Out	172.16.1.1	192.168.3.4	TCP	443	?	?
4	In	192.168.3.4	172.16.1.1	TCP	1357	?	?

Table 2: HTTPS Packet Filtering Analysis

   (b) **(2 marks)** Someone from the outside world (10.1.2.3) attempts to open a connection from port 5150 on a remote host to the Web proxy server on port 8080 on one of your local hosts (172.16.3.4), in order to carry out an attack. Typical packets are as shown in Table 3.

      i. Indicate whether these example packets are permitted or denied and which rule is used in each case.

      ii. Will the attack succeed? Give details.

Packet	Direction	Src Addr	Dest Addr	Protocol	Dest Port	Action	Rule
5	In	10.1.2.3	172.16.3.4	TCP	8080	?	?
6	Out	172.16.3.4	10.1.2.3	TCP	5150	?	?

Table 3: HTTPS Packet Filtering Analysis

(c) **(6 marks)** Let's add a new column for the source port in the rule set. See Table 4. Apply this new rule set to the same six packets of the preceding subsections. Indicate which packets are permitted or denied and which rule is used in each case.

Rule	Direction	Src Addr	Dest Addr	Protocol	Src Port	Dest Port	Action
A	In	External	Internal	TCP	>1023	443	Permit
B	Out	Internal	External	TCP	443	>1023	Permit
C	Out	Internal	External	TCP	>1023	443	Permit
D	In	External	Internal	TCP	443	>1023	Permit
E	Either	Any	Any	Any	Any	Any	Deny

Table 4: HTTPS Packet Filtering Rule Set With Source Port

2. **(10 marks)** Consider the Kerberos Protocol and answer the following questions:

   (a) How do the servers hosting the service verify the authenticity of the client using the ticket the client obtained from the Kerberos protocol?

   (b) How does the authentication server protect a ticket granting ticket from being altered by the client or adversary?

   (c) How are ticket replay attacks prevented? I.e. Mallory observes the network traffic and replays the messages later.

   (d) List the five keys used in Kerberos and the purpose of each key (i.e. what message is it used to encrypt).

   (e) How is the session key distributed by the authentication server?

3. **(10 marks)** Consider the following threats to web security and describe whether and how each is countered by a particular feature of SSL.

   (a) Replay attack: earlier SSL handshake messages are replayed.

   (b) Man-in-the-middle attack: an attacker interposes during key exchange, acting as the client to the server and as the server to the client.

   (c) Password sniffing: passwords in HTTP or other application traffic are eavesdropped.

   (d) IP spoofing: forged IP addresses are used to fool a host into accepting bogus data.

   (e) SYN flooding: many TCP SYN messages are sent to request a connection without responding to fully establish the connection.

2 Programming (40 Marks)

1. **(20 marks)** In the latest twist in Lake Village's gossip circuit, Mr. Goose, the town's beloved private investigator, finds himself at the center of a raging scandal. It all started with a damning front-page exposé in the Columbia Chronicles, which captured Mr. Goose in what appeared to be a rather unheroic retreat from a group of innocent kids during a local football match. The article claims that he instigated the incident by booting a ball away, setting off a chain reaction of chaos and humiliation.

   Naturally, Mr. Goose vehemently denies the accusations, arguing that the article maliciously misinterpreted an old photo from his college days. According to him, the entire affair is a setup orchestrated by none other than his former friend turned arch-nemesis, Mr. Raccoon.

   Enter the town's information forensics squad, tasked with untangling this mess. They've grilled Mr. Raccoon, who, while admitting to harboring a grudge against Mr. Goose, furiously denies any involvement in the article's publication. However, just as things were heating up, the forensics team stumbled upon a juicy piece of evidence: a packet capture straight from Mr. Raccoon's personal computer. Looks like this feud is about to get even messier! Could we be in for a showdown to settle the score?

   Well, there is one way to tell! Your first task is to analyze this packet capture and answer the questions presented below. Justify your answers by presenting all the relevant evidence you can find.

   (a) (1 mark) Looking at the network traces in the file `p-q1-capture.pcap`, do you see anything to suggest that Mr. Raccoon may indeed have been involved in the process of writing the article about Mr. Goose? If so, describe what stands out as interesting. Otherwise, generally describe what you see in the file.

   (b) (3 marks) What are some steganographic techniques that may have been employed for data exchange using the captured packets? Describe at least three possible ways in which data may have been covertly transmitted.

   (c) (5 marks) There is an artifact concealed in the packet capture that can potentially implicate either Mr. Goose or Mr. Raccoon in this investigation. Find this artifact and describe how you found it. What data from the packet capture contains the artifact? How did you analyse the data to discover the artifact?
   (If you could not find the artifact, you can still answer question (c) by describing what data you examined and how you analysed the data.)

   ## Deliverables

   - **(9 marks) Digital forensic report**: your answers to the aforementioned questions (a)-(c).
   - **(3 marks) Evidence artifact:** the conclusive artifact resulting from your investigation. Name this file `recovered`.
   - **(8 marks) Supporting code:** any and all code you wrote as part of your investigation described in part (c). For full marks, include a script that takes the file `p-q1-capture.pcap` as input and outputs a single file `recovered`. Name the script file `get-artifact`.
     Notes:
     i. Your script file `get-artifact` will be both manually graded and executed to ensure it properly extracts the evidence artifact from the packet capture.

ii. While we recommend Python (and Question 2 will require Python), for Question 1 you may code in any popular language such as C, Rust, Bash, or even LaTeX (it is technically possible!) if you so prefer. If you are unsure whether a particular language is acceptable, ask on Piazza.

iii. Be sure to specify in your assignment submission the language you used, including which version of the language. **If there is not enough information to run your code, you will lose up to 5 marks.**

Resources

- For this analysis, you will need the free and open-source program Wireshark. You can get it from wireshark.org. Feel free to explore the program on your own and practice capturing and analyzing your own traffic. We will not ask you to submit any captures of your own traffic, just to look at the packet capture we have provided.
- Following your detailed examination of the network traces provided, you may require a tool capable of parsing packets that you might be interested in dissecting further. We suggest using Scapy, a Python-based interactive packet manipulation program and library. Scapy can decode packets into their respective protocol layers. It provides built-in support for a wide range of protocols, including Ethernet, IP, TCP, UDP, ICMP, DNS, HTTP, and many more. Shown below is an example of using Scapy to parse IP packets from a packet capture:

```
# File example.py

from scapy.all import *

# Function to process each packet
def process_packet(packet):
    if IP in packet:
        ip_src = packet[IP].src
        ip_dst = packet[IP].dst
        print(f"Source IP: {ip_src}, Destination IP: {ip_dst}")

# Read packets from pcap file
sniff(offline="/full/path/to/pcap/file", prn=process_packet)
```

To install Scapy, use:

```
pip install scapy
```

To execute `example.py`, navigate to the directory containing your script and then run:

```
python3 example.py
```

2. **(20 marks)** After assigning the task of analyzing `p-q1-capture.pcap`, the forensics team moved on to investigate Mr. Raccoon's other internet activities. They discovered that Mr. Raccoon had been using the Tor network for almost all his recent online interactions. This meant that his communications were not only encrypted but also anonymized, presenting a significant challenge to investigating which internet services or websites Mr. Raccoon had been communicating with.

   However, they soon realized that devising and training a machine learning model to classify these anonymized Tor traces into specific website categories could be the key. By using this

trained model to classify each of Mr. Raccoon's Tor traces, they could significantly advance their investigation.

As a highly resourceful network security analyst, your expertise is once again needed by the forensics team. In this coding challenge, you will conduct a website fingerprinting attack over Tor. Your primary objective is to build a feature set that enhances the identification of websites browsed through Tor's encrypted tunnel. Below, we introduce the essential concepts you need to master to complete this exercise.

In network systems, traditional encryption hides the content of communications but not metadata like traffic size and direction over time. Low-latency anonymous communication tools like Tor obscure both the content and destination by routing encrypted traffic through several network nodes. However, to maintain an interactive web browsing experience, Tor preserves the packet timing and volume characteristics unique to each web page. This makes Tor vulnerable to traffic analysis attacks, known as website fingerprinting, which aim to uncover users' browsing habits. Website fingerprinting attacks can be seen as a classification problem where an attacker tries to determine if a user visited a specific web page (e.g., a site distributing pirated movies). In a traditionally simple setting commonly known as the closed-world, the attacker monitors *n* web pages, and the user is assumed to visit one of these *n* pages. To set up a website fingerprinting attack, the attacker first collects traffic traces from each of the *n* web pages by making web requests. The attacker then extracts a feature set from the traffic data and trains a model to identify the websites based on these features. When the user visits a web page, the attacker collects the traffic trace and uses the model to predict which of the *n* pages the user visited.

- **Goal:** In this exercise, you will perform the closed-world website fingerprinting attack using a dataset containing Tor traffic traces. We provide sample Python code to demonstrate these attacks with a basic feature set. Your task is to enhance this feature set to improve the attacks' effectiveness and answer some related questions based on your findings.
- **Dataset:** The provided dataset includes 90 instances for each of 100 monitored pages. These 100 monitored pages come from a list of blocked web pages in China, the UK, and Saudi Arabia, and are listed in `/p-q2/data/knnsitelist.txt`.
- **Dataset Structure:** Instead of raw packet traces, the dataset contains preprocessed Tor cell sequences for page loads. Tor sends data in 512-byte encrypted cells to hide traffic patterns. In this dataset, each cell sequence for a monitored site is stored in a file named *W-L*, where *W* is the webpage ID and *L* is the page load instance (e.g., 0-4 is the fourth load of monitored webpage 0). Cell sequences for non-monitored sites are stored in files named with the webpage ID only. Each file contains rows in the format $t_i < tab > s_i$, where t_i is the time and s_i is the size of the i-th cell. Since each Tor cell is 512 bytes, s_i indicates direction: +1 for outgoing (client to website) and -1 for incoming (website to client). You will find Tor traces for your closed-world analysis in `p-q2/data/closed_world`.
- **Classifier:** To perform website fingerprinting attacks with the provided dataset, use the classifier algorithm in the `/p-q2` folder. The main program, `classify.py`, works in two stages: training and testing.
  - **Training Stage:** Uses 80% of the data to build a model for each website's traffic. Monitored websites get specific classes.
  - **Testing Stage:** Uses the remaining 20% to test the model by predicting the class of website traffic. Accuracy is evaluated by comparing predictions with actual data.

In general, there are many ways to implement a classifier. For this exercise, we have implemented a classifier using the k-Nearest Neighbors (k-NN) algorithm which is a supervised machine learning method. Essentially, the k-NN classifier calculates the distance between a cell sequence *P* from the training set and its *k* nearest neighbors within the same set. For $k = 1$, the object is classified based on the class of its single nearest neighbor. For $k > 1$, *P* is classified by the majority vote of its *k* nearest neighbors. Figure 1 illustrates this concept for $k = 3$ with two different classes (blue squares and red triangles).

The k-NN algorithm uses a distance metric d to measure the similarity between two cell sequences. This distance is determined by various characteristics, known as features, which describe a data point. A feature could be as straightforward as "the total number of cells within a cell sequence." Multiple features can be combined into a feature set $F = f_1, f_2, f_3, ..., f_n$, where each feature f is a function that takes a cell sequence P and computes $f(P)$. Ideally, we choose features so that cell sequences from the same webpage have similar features, while those from different webpages do not. You will use a modified version of k-NN that adjusts the weight of each feature according to its importance in distinguishing different classes. More informative features will have higher weights in distance calculations, thus having a greater impact on classification results. The effectiveness of the classifier heavily depends on the selected features and their respective weights. Formally, our k-NN classifier works as follows: It starts with a training set $S_{train} = P_1, P_2, P_3, ..., P_n$, where each point is a cell sequence with a class label $c(P_i)$ that identifies a monitored website or a group of non-monitored websites. Given a testing point P_test, the classifier computes the distance $c(P_{test}, P_{train})$ for each P_{train} in S_{train}. The algorithm then labels P_{test} based on the classes of the k closest training points.

- **Classifier Setup:** Uses 80% of monitored page samples (7200) for training and 20% (1800) for testing.
- **Performance Metrics:** In a closed-world attack, a user is assumed to be able to visit a particular website among n monitored websites. When measuring the effectiveness of this attack, we are mostly interested in its accuracy (ACC), that is, the fraction between the number of correct predictions over the total number of predictions. An effective attack should exhibit a large accuracy, meaning that the majority of accesses to monitored pages are successfully identified.

Setup: In order to use the provided Python scripts, you must first ensure the packages in `requirements.txt` are installed. We recommend using a virtual environment, as follows:

```
# setup
python3 -m venv venv  # this creates a directory named 'venv'
source venv/bin/activate # 'venv/bin/activate' is the path to an
    activation script
pip install --upgrade pip
pip install -r p-q2/requirements.txt
```

After you are finished using the Python scripts, use the `deactivate` command to exit the virtual environment:

```
deactivate
```

If you want to use the Python scripts again, first re-activate the virtual environment:

```
source venv/bin/activate  # use the path to the same activation script
    from above
```

Extracting traffic features: Use `/p-q2/code/fextractor.py` to extract features from the dataset and save the output to a folder as shown in the example below:

```
# feature extraction
cd p-q2/code
python3 fextractor.py --traces ../data/closed_world --out ../data/
    features
```

where `--traces` should point to the folder containing the Tor cells dataset, and `--out` to the folder where to save the extracted features.

Classification: Use `/p-q2/code/classify.py` to issue a website fingerprinting attack over the provided Tor traffic as shown below:

```
# closed - world attack
python3 classify.py --features ../data/features --k 5 --out res_k5
```

where `--features` points to the folder containing the features obtained in the previous step, `--k` selects the number of neighbours used when predicting a label, and `--out` points to the location where to save the file containing the classification results.

You may observe poor results when running these examples. This is because the feature set used by the classifier is not adequate to accurately differentiate between various page loads.

Using the provided dataset and classifier code, perform the following exercises. Note that you **must not** alter any of the hardcoded parameters of the classifier in your experiments, such as the proportion 80% / 20% between training and testing set sizes.

(a) Improve the accuracy result of the closed-world attack, when using $k = 5$ neighbours, by augmenting your feature set. You can do this by generating multiple descriptive characteristics of Tor cell sequences, for instance based on their time and direction. You may increase the feature set of the classifier by adding new features in `/p-q2/code/fextractor.py`'s extract function. Read the comments in the code for more details on how to add new features. Aim for the highest possible accuracy.

To re-run your website fingerprinting attack (i.e., the `/p-q2/code/classify.py` script) with new features, remember to re-run `/p-q2/code/fextractor.py` first every time you update the feature set! This will generate newer features that will in turn be used to train the classifier.

(b) Using your best feature set, analyze the influence of the choice of k neighbours in the accuracy of the closed-world attack.

Deliverables

- **(10 marks) New features:** Specify the new features you added, and explain your rationale for choosing each one. Provide reasonable explanations for how they contribute to increased classification accuracy. You are free to add more than 5 new features to the set. You will receive **2 marks** for each feature added to the existing set, up to a maximum of **10 marks** for this section. You may use a table similar to Table 5 to detail your reasoning.
- **(4 marks) Best feature set and accuracy:** Indicate your best feature set and report the highest classification accuracy achieved with it.
- **(6 marks) Influence of neighbors:** First, build a table similar to Table 6 where each cell represents the accuracy obtained by the classifier when running the closed-world attack with k = 1, 2, 5, 10, 15. Can you spot a trend in the accuracy of the classifier as k increases? Explain the variations observed.

Table 5: Feature Set Analysis

	Feature	How does this feature increase accuracy?
1		
2		
.		
.		
5		

Table 6: Influence of k on classification accuracy

k	1	2	5	10	15
ACC					

- **Code:** Please submit the file `fextractor.py` located in the directory `/p-q2/code`, which should include the extended feature set implemented within the `extract()` function. Your submission will undergo testing in our grading environment to validate the reported accuracies. Failure to submit the code or submitting non-functional code may result in a penalty of **5 marks** deducted from this question.